INDEX

NAME Nanda Kishore D CSE - III year SUBJECT

STD. DIV. ROLL NO. SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1)	22/7	STUDY OF NETWORK COMMANDS		25/8/26
2)	27/7	Study of different types of network cable.	9	9
3)	04/8	Cisco packet tracer.	9	4/8/26
4)	10/8	Wireless LAN	9	10/8
5)	18/8	wireshark	9	18/8
6)	~~15/09~~	Error detection & correction using hamming code.	9	7/9
7)	~~22~~ 21/09	Nmap to discover live host using tryhackme platform	9	21/9
8)	28/9	End to End communication ~~at~~ at transport layer.	9	28/9
9)	28/9	sliding windo protocol	9	28/9
10)	6/10	subneting in cisco packet tracer	9	6/10
11)	6/10	PING PROGRAM	9	6/10
12)	12/10	RAW Sockets to implement packets.		
13)	12/10/26	Routing at network layer	9	12/10
14)	12/10/26	Internetworking with routers.	9	12/10

EX NO: 1
22/7/26

STUDY OF NETWORK COMMANDS

Aim:

To Study of various network commands used in Linux and windows

arp -a:

ARP is short form of address resolution protocol. it will show the IP address of your computer along with the IP address and MAC address of your router.

hostname:-

This is the simplest of all TCP/IP commands it simply, display the name of your computer.

ipconfig /all:

This commands displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP and type of Ethernet adapter in your System.

Nbtstat -a:-

This commands helps solve problems with net BIOS name resolution. (Nbt stands for Net Bios over TCP/IP)

Netstat:

netstat displays a variety of statistics about a computer active TCP/IP connections both incomming and outgoing as well as viewing routing tables

NSlookup :-

(name server lookup) is a tool used to perform DNS lookups in linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS server of a domain. NSlookup can operate in two mode.

eg:- nslookup www.google.com

Pathping :-

Pathping is unique to windows and is basically a combination of the ping and tracert commands. Pathping trace the route to the destination address then branches as OS Second of each router along the way gathering statistics on the rate of data loss along each hop

Ping :-

(Packet internet gramber) commands is the best way to test connectivity between two nodes. Ping use ICMP to communicate to other device.

1) # Ping hostname (Ping localhost)
2) # Ping IPaddress (Ping 4.2.2.2)
3) # Ping fully qualified domain name (Ping www.facebook.com)

Route :-

route command is used to show / manipulate the IP routing table it is primarily used to set static routes to specific host or networks via an interface.

Ubuntu implement linux networking networkmanager

)))

The IP command is one of the basic command, Easy administration, will used in daily work. from setting up network systems and assigning IPs to troubleshooting existing systems and assigning IPs. tool to showing existing systems

IP <options> <OBJECT> <commands>

a) To show the IP address on an interface on your server:

a) [root@server ~]# IP address show

b) To assign a IP to an interface, for example,

eps03:

a. [root@server ~]# IP address add 192.168.1.25/24 dev enps03

c) To delete an IP on an interface:

a. [root@server ~]# I paddress del 192.168.1.254/24 dev enps03

d) After the states of the interface by bringing the interface into

online: [root@server ~] ip link set the up.

e) After the status of the interface by ~~bringing~~ the interface into

~~Offline~~: [root@server ~]# IP link set ethe down

f) After the states of the interface by enabling promiscuous mode for.

ethe: [root@server ~]# ip link set ethe

g) Add a default route (for all addresses) via the local gateway 192.168.1.254 that is reached on device eth0

[root@server ~]# IP address route add default via 192.168.1.254 dev eth0

h. add a route to 192.168.1.0/24 via the gateway at 192.168.1.254:

[root@server ~]# ip route add 192.168.1.0/24 via 192.168.1.254.

i) add a route for 192.168.1.0/24 via the gateway at that number reached on device eth0 : [root@server ~]# ip route add 192.168.1.0/24 dev eth0

j) Delete the route for 192.168.1.0/24 via the gateway at 192.168.12.254.

[root@server ~]# ip route delete 192.168.1.0/24 via 192.168.1.254.

k) display the route taken for IP 10.10.1.5

[root@server ~]# IP route get 10.10.14

8) Ifconfig :-

The ifconfig command was in staple in many system's tool belt for configuration and troubleshooting networks. It has since been replaced by the IP command discussed above.

5. MTR

MTR (Matt's traceroute) is network with a command-line interface that serves as a network diagnostic and troubleshooting

[illegible] command combines the functionality of the ping and traceroute commands. Just like traceroute, the mtr command will show the route taken by packets to the specified host. mtr starts by sending packets to each router on the network, with increasing TTL values. The output will then display the packet loss and latency at each hop. With this information you will get more information about the issue and use this to see troubleshoot during slow network. If you see sudden increase in response time of the segment of the network is a failure.

mtr <options> hostname/IP.

a) The basic mtr commands shows you the statistics including each hop (hostname) with time and loss %. [root@server~]# mtr google.ca

b) shows numeric IP address (if you use. you will get IP address (numbers) instead of hostname

[root@server~]# mtr -g google.com

c) Show the numeric IP address and host names too: [root@server~]# mtr -b google.com.

d) set the number of pings that you want to send: [root@server~]# mtr -c 10 google.com.

4. tcpdump:

The tcpdump commands is designed for capturing and displaying packets.

You can install tcpdump with the command below:

[root @ server ~]# dnf install -y tcpdump

before starting any capturing you need to know which interface can yours will need to used sudo or have root access in this case I

[root @ server ~]# tcpdump -D.

if you want to capture traffic on eth0 you can initiate that with tcpdump -i eth0

Sample output:

[root @ server ~]# tcpdump -i eth0.

[root @ server ~]# tcpdump -i eth0 -c10

capture traffic to and from one host you can filter out traffic coming from a specifically host. For example if you find traffic comming from and going to 8.8.8.8 use the command.

[root @ server ~] tcpdump -I eth0 -c 10 host 8.8.8.8

For traffic comming from 8.8.8.8 use:-

[root @ server ~]# tcpdump -i eth0 src host 8.8.8.8

For outbound traffic going to 8.8.8.8 use.

[root @ server ~] # tcpdump -i eth0 dst host 8.8.8.8

capture traffic to and from a specific network using the command

[root @ server ~]# tcpdump -i eth0 net 10.10.0.

255.255.255.0

Or :

[root@server ~]# tcpdump -i eth0 net 10.1.0.

Capture traffic to and from port numbers

Capture only DNS Port 53 traffic

[root@server ~]# tcpdump -i eth0 host 8.8.8.8 and port 53

To capture only HTTPS traffic

[root@server ~]# tcpdump -i eth0 -c 10 host www.google.com and port 443

To capture all port except port 80 and 25:

[root@server ~]# tcpdump -i eth0 port not 53 and not 25.

Result:-

Thus the study of various network commands used in linux and windows completed.

28/2/24

Aim :-

Study of different types of network cables

a) understand different types of network cable
different types of cables or network cable

1) Unshielded Twisted pair (UTP) cable
2) shielded Twisted pair (STP) cable
3) coaxial cable
4) Fibre optic cable.

Cable Type	category	Maximum data Transmission	Advantages / Disadvantages	Application / use
UTP	category 3	10 bps	Advantages: * cheaper in cost * Easy to install as they have a smaller overall diameter. Disadvantages:- * More prone to (EMI) Electromagnetic interfare and noise	10 base-T Ethernet
	category 5	upto 100 MbPS		Fast Ethernet gigbit ethernet
	category 5e	1000bps		Fast ethernet gigabit ethernet
STP	Category 6, 6a	10Gbps	Advantages:- * Shielded. * Faster than UTP * less susceptible to noise and Interface. Disadvantage:- * Expensive * greater installation effort.	Gigabit ethernet, 10G ethernet (55m) widely used. in data centres
STP	Category 7	10 Gbps		Gigabyte ethernet, 10 G ethernet (100m)

Cable Type	category	maximum data transmission rate	Advantages/ Disadvantages	Application use.
Co-axial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages → High bandwidth → Immune to interference → low loss bandwidth → versatile Disadvantages → limited distance → cost → size is bulky	speed of signal in room Television network High speed Internet connection.
Fibre optics cable	single Mode Multi Mode	100 Gbps	Advantages → High speed → High bandwidth → High security → long distance Disadvantages → expensive → Requires skilled installers	maximum distance of fibre optics cable is around 100 metres.

Result:-

Thus the different types of network cables are studied successfully.

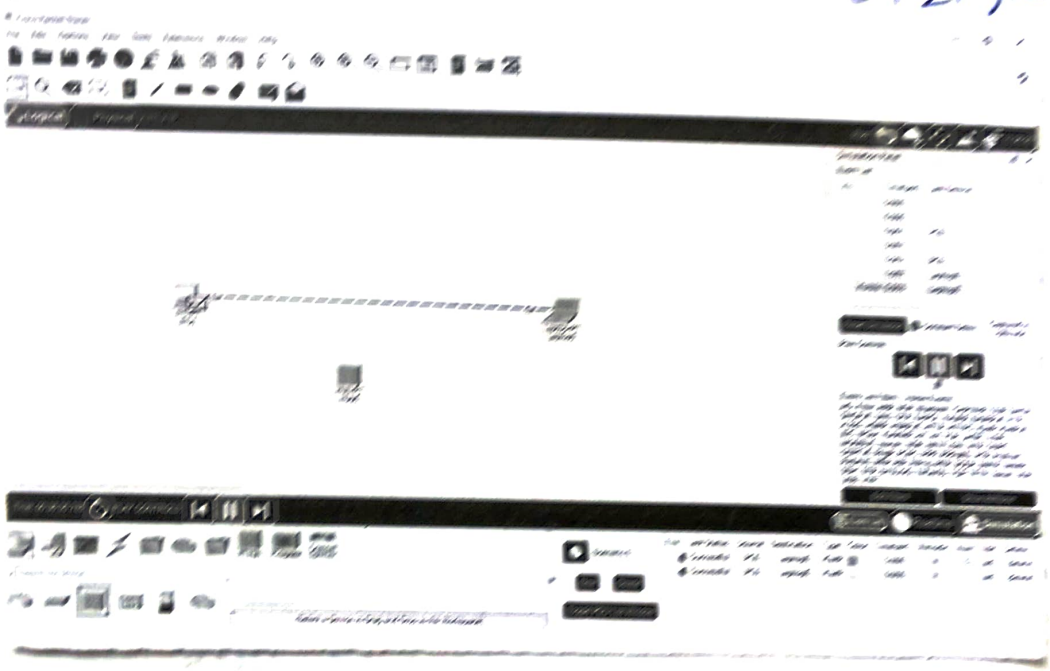
4/8/25

Exp: 6 Experiments on circuit simulation tool (Cisco packet Tracer simulation tool)

Aim:

To understand its environment and behavior of network devices in circuit packet tracer simulator.

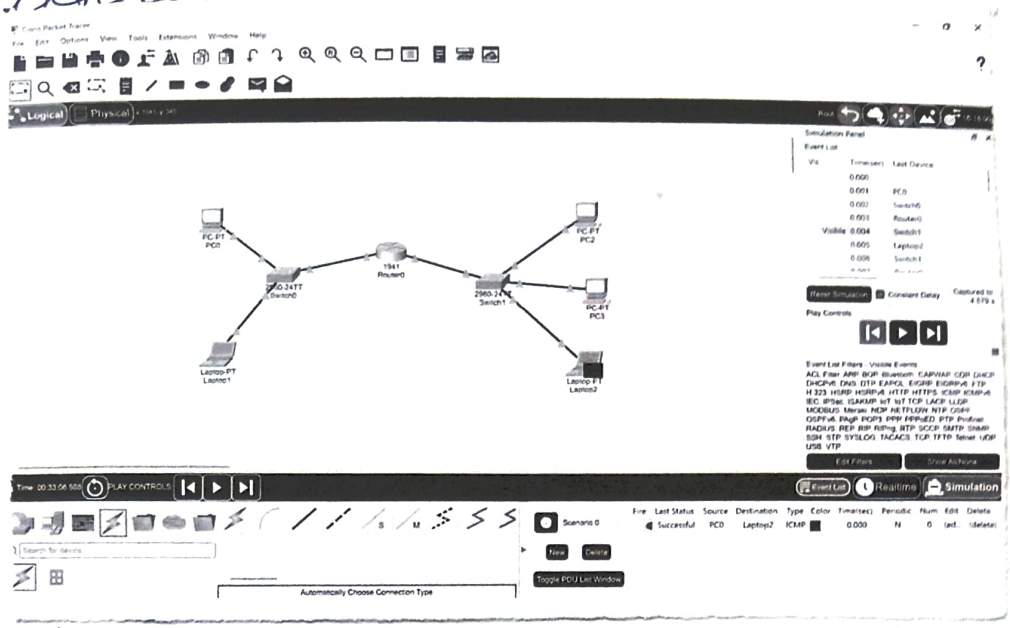
1) Peer - to - peer connection (P2P):-



Cross - over cables were used to connect the PC and the laptop. only a cross over cable can enable communication directly between same type of end - devices.

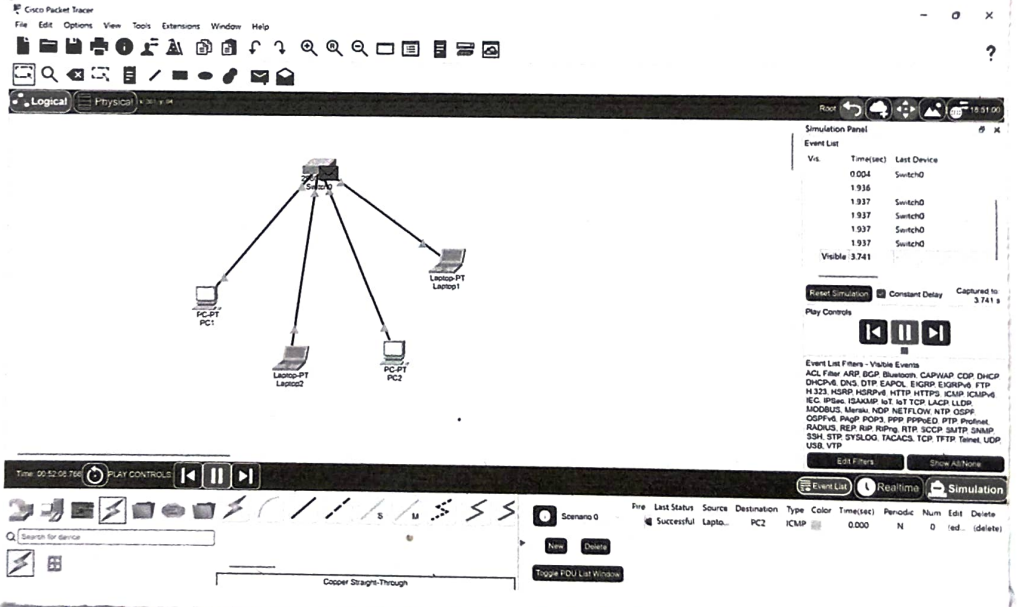
Usage of such cables have been replaced by switches, hubs and routers

2) Hub connection :-

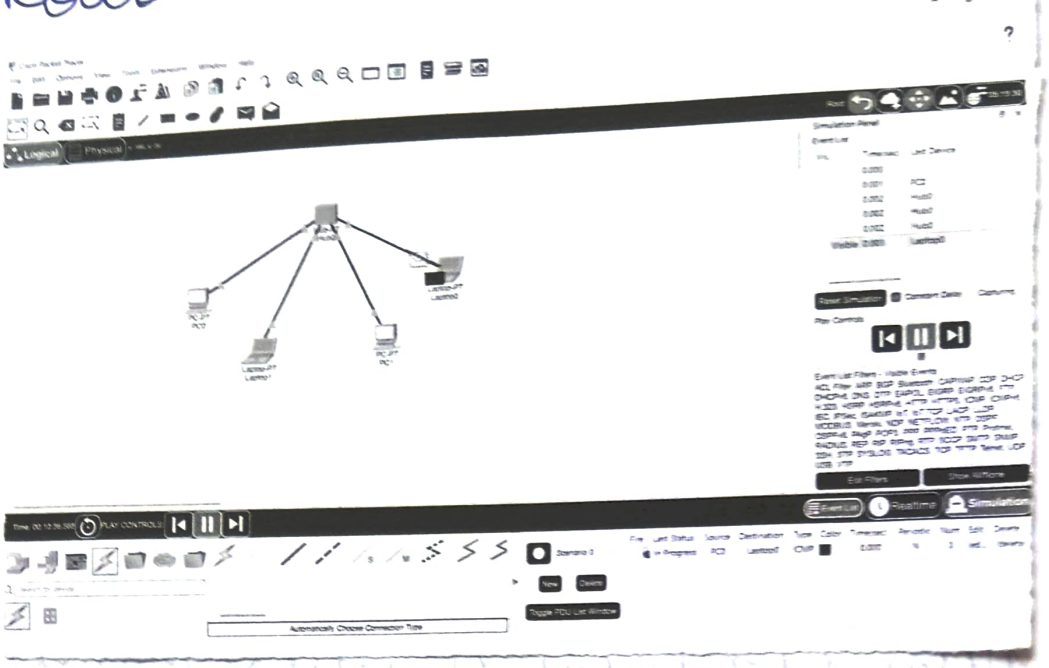


A hub connect multiple device in a network by blindly broadcasting all incoming data signal to every connected devices.

3) Switch connection :-



Switch connect multiple device within single LAN and intelligently forward Data packets directly to specific destination using MAC address



Q

A router connects different subnets or distinct networks together, directly data traffic between them using IP address and default gateways.

Result :-

Hence the environment of CISCO Packet Tracer and behaviour of all the network devices in the software was studied. simulated and verified successfully.

02/08/24 EXP 1: Setup and configure a LAN using Switch and ethernet cables.

What is LAN?

LAN stands for local area network which is a group of computers and connected devices linked together in a small physical location like a, home, school, or office building.

How to setup LAN :-

Step 1 :- connect the hardware

→ place your router in a central location near a power outlet.

→ connect your modem to the router's WAN or internet port using an Ethernet cable.

→ Plug the router into power and wait for it to turn on.

Step 2 :- connect your devices

→ Wired setup :- Plug one end of a Ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router.

Step 3:-
Configure the router.

→ Type your IP address (192.168.1.1 or 192.168.0.1) into the address bar and press enter.

→ Log in using the default username and password printed on the router.

Step 4:- Check the connectivity b/w switch and other machine by using ping command in the command prompt of the device.

Step 5:- Try to access the shared folder from others computers of the network.

Result:-

Wireless LAN using switch and ethernet cables are executed succesfully in cisco packet tracer.

18/08 Exp: 5 Wireshark

Aim :-

Experiments on packet capture tool wireshark.

Procedure :-

* Select local area connection in wireshark.
* TO capture option
* select stop capture automatically after 100 packets.
* Then click start capture
* Search TCP packets in search bar.
* TO see flow graph click statistics flow graph
* save the packets

1) create a filter to display only TCP/UDP packets, inspects the packets and provide the flow graph.

* Select LAN connection in wireshark
* TO capture option
* select stop capture automatically after 100 packets
* Then click start capture
* Save the packets

2) Create a filter to display only ARP Packets and inspect the packets

Procedure:-

* Go to the capture option
* Select Stop capture automatically after 100 packets.
* then click start capture.
* search ARP packets in search bars.
* save the packets

3) create a filter to display only HTTP Packets and inspect the packets

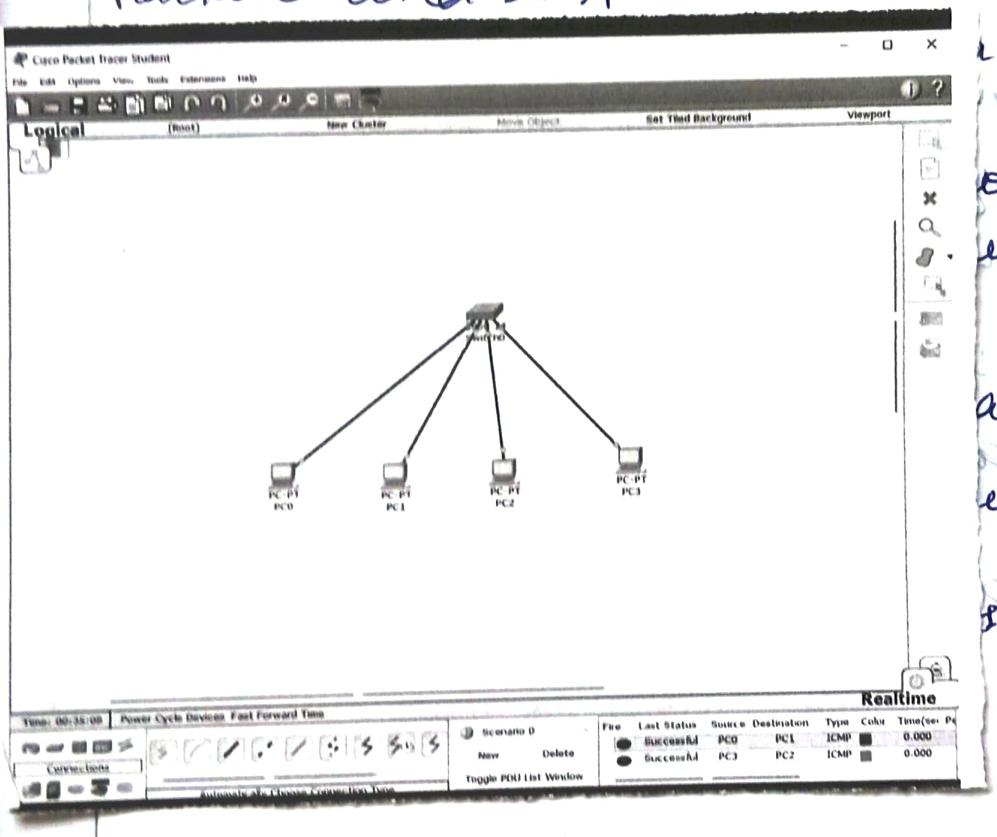
Procedure:-

* Select local area connection in wireshark.
* Go to capture option
* Select stop capture automatically after 100 packets.
* then click start capture
* Search HTTP packets in search bars.

4) create a file to display only DNS packets and inspect the flow graph.

* Go to capture option
* Select stop capture automatically after 100 packets
* then click start capture.
* search DNS packets in search bars.
* To see the flow graph click statistics flow graph.

5) Create a filter to display only DHCP packets and inspect the packets.



Result:-

Wireshark - on packet capture tool experiment are learned and executed succesfully.

7/9/22

EXP: 6 Error detection & correction using Hamming code.

Aim:

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Sender :-

* Input to sender file should be a text of any length program should be convert the text to binary.

* Apply hamming code concept on the binary data and add redundant bits to it.

* Save this output in a file called channel.

Code:-

```
data = input("enter 4 bits : ")
d1, d2, d3, d4 = map(int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
cod = [P1, P2, d1, P4, d2, d3, d4]
```

Print ("sender hamming code:", "join (map(str, code))).

Receiver side:-

* Receive the hamming code from the Sender.
* Enter the received code.
* calculate the parity check values.
* Find the error position using the parity values.
* correct the error by changing the bit
* Display the error by changing the bit.

~~* D~~

code:-

r = list (map (int, input ("enter received 7bits: ")

$r_1 = r[0] \wedge r[2] \wedge r[4] \wedge r[6]$

$r_2 = r[1] \wedge r[2] \wedge r[5] \wedge r[6]$

$r_3 = r[3] \wedge r[4] \wedge r[5] \wedge r[6]$

error = r1*1 + r2*2 + r4*4.

if error = 0:

Print ("NO error").

else:

Print ("Error at position:", error).

r [error-1] ^ = 1

Print ("corrected code:", "join (map(str, received)))

Inputs:

Number of data bits : 16

Number of parity bits : 5

Received data bits : 0101011

Outputs:

Error at position: 3

Corrected word : 0101001

Result:-

Error detection & error correction using hamming code was executed successfully.

EXP: 7 NMAP to discover live Host using Nmap scans on the tryhackme problem.

22/9/26

Aim :-

To discover live hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the tryhackme platform room link : https://tryhackme.com/room/nmapo/

Procedure :-

Task :- 1 Introduction

*ARP Scan: This scan uses ARP request to discover live hosts.

* ICMP Scan: This scan uses ICMP requests to identify live hosts.

* TCP/UDP ping Scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

Task :- 2 Subnetworks

Network segment - It is a collection of computers linked together through a common "medium".

Subnetwork - It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an ip network.

* Subnets with /16
* Subnets with /24

Task :- 3 Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action".

1) list
   - <MACHINE-IP> <scanme.nmap.org> <example>
   - will scan 3 IP addresses.

2) Range
   - 10.11.12.15-20.
   - will scan 6 IP addresses

3) subnets
   - <MACHINE-IP>/30
   - will scan 4 IP address.

nmap -sL <TARGETS>

Task :- 4 Discovering live hosts.

1) ARP, from link layer - commonly used for man-in-middle Attack (ARP spoofing and ARP poisoning).

2) ICMP from networks layer - commonly used in DDOS attacks.

3) TCP from transport layer - SYN flooding, TCP reset, TCP session hijacking attacks.

4) UDP from transport layer - Also used in DDOS Attacks.

Task :- 5 Nmap host discovering using ARP

1) when a privileged user tries to scan LAN (Local area network), Nmap uses ARP request.

2) To discover internal hosts without port scanning - live systems use nmap -sn ; targets.

3) nmap -PR -sn <TARGETS>

-PR specifies that you only want an ARP scan.

4) nmap -PR -sn <MACHINE IP>/24

5) nmap -PR -sn <ATTACKBOX IP>/24.

Task 6 :- Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo request to discover live hosts.

2) nmap -PE -sn TARGET to check host is live.

3) -PP options instruct Nmap to use ICMP timestamp request.

4) -PM option can enable this scan to ICMP address mask request.

Task 7 :- Nmap Host discovery using TCP and UDP

1) Nmap to use TCP & YN ping use -PS followed by the port number, range, list or a combination of this options.

-PS21, -PS25 target ports 21, 22, 23, 24, 25.

2) nmap -PS -sn TARGET check the TCP port is open.

22/9/25

Result :-

Live host discovery using Nmap on the TryHackMe Platform successfully executed.

28/9/24 EXP:8 End to End communication at Transport layer

a)

Aim:-

Implement echo client server using TCP/UDP Packets.

Procedure:-

* Create a socket using python socket programming.
* specify the IP address and port number
* Bind the server socket to the IP address and port.
* Make the server listen for client connections
* connect the client to the server.
* send and receive messages between client and server.
* Display the received message
* Close the socket connection.

Code:-

TCP-server:-

```
import socket.
Server = Socket.Socket(socket.AF_INET, Socket.SOCK_STREAM).
Server.bind("127.0.0.1", 5000))
Server.listen(1)
Print("server is waiting for connection...")
Conn.addr.= Server.accept()
```

```
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("client : ", data)
    conn.send(data.encode())
    conn.close()
server.close()
```

TCP - Receiver client.)

```
import socket
client = socket.socket(socket.AF_INET socket.
SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("enter message: ")
client.send(message.encode())
reply = client.recv(1024).decode()
print("server : ", reply)
client.close()
```

Terminal-1 server.

```
Python tcp-server.py
server is waiting for
connection ...
Client connected.
client : Hello
```

Terminal 2 :- client

```
Python tcp_client.py
enter message : Hello
Server : hello
TCP echo is working
```

Result :-

The TCP/UDP packets using echo client server socket program is executed succesfully

b)

Aim:-

Implement chat client server using TCP packets.

Code:-

Chat_server side :-

```
Import socket.
Server = socket.Socket (socket.AF_INET,
    socket, socket_STREAM).
Server.bind [("127.0.0.1", 6000)]
server.listen (1)
Print ("waiting for client...")
conn, addr = server.accept()
print ("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye".
    Print ("client ended chat")
    break.
    Print ("client:", message)
    reply = input ("server:").
    conn.send (reply.encode())
    if reply.lower() == "bye":
conn.close()
server.close()
```

Chat_client side:-

```
import socket.
client = socket.socket(socket.AF_INET,
  Socket.Sock_STREAM)
client.connect(("127.0.0.1", 6000))
Print ("connected to server")
while True:
    message = input("client:")
    client.send(message.encode())
    if message.lower() = "bye":
      break.
    reply = client.recv(1024).decode()
    Print ("server:", reply)
    if reply.lower() == "bye":
      break.
client.close()
```

Terminal 1:-

```
Python Chat_server.py
waiting for client...
client connected
Client: hello
Server: Hi
Client: How are you?
Server: Fine
```

Terminal 2:-

```
Python chat_client
connected to server
Client: Hello
Server. Hi
Client: How are you
Server: Fine.
```

Result:-

29/9/25 The chat programming using sockets by TCP client & server was implemented successfully

28/9/24

Exp:-9 Sliding window protocol.

Aim:-

write a program to implement flow control at data link layer using sliding window protocol. simulate the flows from the one node to another.

Code:-

```
import time
window = int(input("enter window size:"))
message = input("enter message:")
start = 0
while start < len(message)
Print("SENDER")
end = start + window
if end > len(message):
    end = len(message)

#sender buffer.

    sender_buffer = [ ]
    for i in range (start, end)
    frame = (i, message[i])
     sender_buffer.append(frame)
     Print("FrameNo:", i, "Data:", message[i])
    Print("\nsender buffer:")
    Print(sender_buffer)
```

```
# delay
    Print ("\n Waiting for receive ...")
    time. Sleep(1)

# Receiver
    Print ("RECEIVER")
    expected = start
    error = false.
    for frame in sender_buffer:
    frame_no = frame[0]
    data = frame[1]
    Print ("received frame:", frame_no,
           "DATA:", data).
    If frame_no = expected:
        expected = expected + 1
    else:
        error = True.
        break.

# Receiver buffer.
    if error:
        receiver_buffer = ("NACK", expecte
        Print ("\n receiver Buffer:")
        Print (receive_buffer)
    else:
        ack = expected - 1
        receiver_buffer = ("ACK", ack)
        Print ("\n receiver buffer:")
        Print ("receiver_buffer !!).
```

#sender reads ACK / NACK

```
Print ("In sender (Waiting Response")
response = receiver_buffer[0]
number = receiver_buffer[1]
if response == "ACK":
    Print ("ACK Received : ", number)
    start = number + 1
else :
    Print ("NACK received : ", number)
    start = number.
Print ("TRANSMISSION COMPLETED").
```

Output :-

Enter window size : 3
Enter message : Hello.

SENDER

Frame NO : 0 DATA : H
Frame NO : 1 DATA : e
Frame NO : 2 DATA : l.

[(0, 'H'), (1, 'e'), (2, 'l')]

Waiting for receive...

Receiver

Received Frame : 0 DATA : H
Received Frame : 1 DATA : e
received Frame : 2 DATA : l
Receiver Buffer :
('ACK', 2)

Sender

Frame NO : 3 DATA : l
Frame NO : 4 DATA : o
Sender Buffer :
[(3, 'l'), (4, 'o')]
waiting for receive.

Receiver

Receive fame : 3 DATA : l
Receiver fame : 4 DATA :
Receiver buffer :
('ACK', 4)

Sender checking response
ACK received : 2

Sender checking response
ACK Received : 4

Transmission completed.

Result :

Sliding window protocols are learned and executed successfully.

28/9/24

EXP: 16 Subnetting in CISCO Packet Tracer

Aim:

To implement classless IP subnetting using Cisco packet Tracer and verify the connectivity between different subnets.

Requirements:-

* Cisco packet tracer
* Routers
* switches
* PCs
* Ethernet cables.

Procedure:-

1) Open cisco packet tracer
2) create a new network topology
3) Add routers, switches and PCs to the workspace connect the device using ethernet cables.
4) divide the network into small subnets using the /27 subnet mask.
5) configure the IP address and subnet mask of the router interfaces.
6) configure the IP address, subnet mask and default gateway for each PC.
7) configure the required switch Ports.
8) Enable the router interface using the no shutdown.
9) Verify that the duties in the network communicate

IPaddressing:-

	IP address	subnet Mask	default gateway
PC-PT PC0	199.168.2.12	255.255.255.192	192.168.3.1
PC-PT PC1	192.168.2.11	255.255.255.192	192.168.3.1
PC-PT PC2	192.168.1.12	255.255.255.192	192.168.1.1
PC-PT PC3	192.168.1.11	255.255.255.192	192.168.1.1
PC-PT PC4	192.168.2.14	255.255.265.192	192.168.4.1
PC-PT PC5	192.168.2.13	255.255.255.192	192.168.4.1
PC-PT PC6	192.168.1.14	255.255.255.192	192.168.2.1
PC-PT PC7	192.168.1.13	255.255.255.192	192.168.2.1

Result:-

Thus, classless IP subnetting was successfully implemented using cisco packet tracer, and connectivity between the network device was verified sucessfully.

EXP: 11 PING PROGRAM

Aim:-

To implement a ping program using socket programming to check the connectivity between two network devices

Program:-

```
import socket
import time
import sys
def UDP_ping_server(host="127.0.0.1",
                    Port=12,000)
    server_socket.bind((host, port))
    Print(f"[server] listening on {host}:{Port
    while True:
        msg, client_address = server_socket.
                                recvfrom(12
        Print(f"[server] Received '{msg.decode()
                from {client_address}").
        Server_socket.sendto(msg, client_addr
def UDP_ping_client(server_host="127.0.0.1",
ServerPort=12,000, count=5):
    client_socket.settimeout(1)
    for i in range(1, count+1):
        msg = f"Ping {i} {time.time()}"
        Start = time.time()
        client_socket.sendto()
            msg.encode(), (server_host, server_
Port)
```

```
try:
    data, _ = client_socket.recvfrom(1024)
    end = time.time()
    rtt = (end - start) * 1000
    print(
        f"Reply from {server_host}: {server_port}"
        f"{data.decode()} | RTT = {rtt:.2f} ms")
except socket.timeout:
    print(f"request {i} timed out")
client_socket.close()
if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        udp_ping_client()
```

Output:

```
Python udp_ping.py server
[server] listening on 127.0.0.1:12000
[server] received 'Ping 1 1791476483.6564088' from
('127.0.0.1', 58522)
Python UDP_ping.py
    Reply from 127.0.0.1:12000/Ping 1 1791476483.
                                    6564088/RTT
    = 0.92 ms.
Reply from 127.0.0.1:12000/Ping 2 1791476483.
                                    6980255/RTT
= 0.25 ms.
```

Result:-

Thus, the ping program is successfully executed.

6/10/26

EX NO: 12 Write a code using RAW Sockets to implement packet sniffing

19/10/26

Aim:-

To write a python program using RAW sockets to capture and analyze network packets.

Algorithm:

* start the program.
* create a raw socket using socket.AF_INET and socket.SOCK_RAW
* capture incoming network packets
* extract the ethernet header from each packet.
* Identify the source MAC address, destination MAC address and ethernet protocol
* Display the packet details.
* Repeat until the program is stopped.

Code:-

```
from scapy.all import sniff
from scapy.layers - call back (packet):
    if IP in packet:
        ip = layer = packet[IP]
        protocol = IP_layer.proto
        src_ip = ip_layers_src
        dest_ip = ip - layer.dest
```

```
# determine the protocol
protocol_name = " "
if protocol == 1:
    protocol = name = "ICMP"
elif protocol == 6:
    Protocol_name = "TCP"
elif protocol == 17:
    Protocol_name = "UDP"
else:
    Protocol_name = "unknown protocol"
Print(f"protocol : {Protocol_name}")
Print(f"Source IP : {src_ip}")
Print(f"Destination IP : {dst_ip}")
Print("_" * 50
Sniff( if ace = "wifi", Prn = Packet_callback,
                        filter = "ip", store = 0)
```

output :-

```
Destination IP : 224.0.0.251
Protocol : TCP
Source IP : 172.16.11.15
```

```
Router>enable
Router#show ip route static
S     30.0.0.0/8 [20/0] via 40.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S     30.0.0.0/8 [10/0] via 20.0.0.2
S     50.0.0.0/8 [10/0] via 40.0.0.2
S     60.0.0.0/8 [1/0] via 40.0.0.2
```

12/10/23

messages were successfully sent and received between client and server.

EX: 13
12/10/24

Rating at network layers

a) Simulate static protocol configuration using CISCO packet tracer.

Aim:-

TO simulate a static routing configuration using CISCO Packet Tracer.

Procedure:-

* Open cisco packet tracer and create the given network topology
* Connect 3 routers and PCs using suitable cables.
* Configure IP address for the routers and PCs.
* Enable all router interface using no shutdown.
* Configure static routes on Router 0, Router 1, and router 2 using IP mobile.
* Set main and backup router using different administrative.
* Disconnect the main link and verify that the Backup route works.
* Thus a static routing is successfully configured and verified..

```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    11 ms     8 ms      5 ms      192.168.1.250
  3    29 ms     26 ms     25 ms     192.168.1.245
  4    22 ms     28 ms     20 ms     20.0.0.2

Trace complete.
```

12/12/22

Result :-

RIP successfully configured and dynamic routing between PC0 and PC1 was verified.

Design and configure an internet Reboot
in your using switches, router and
Ethernet cables.

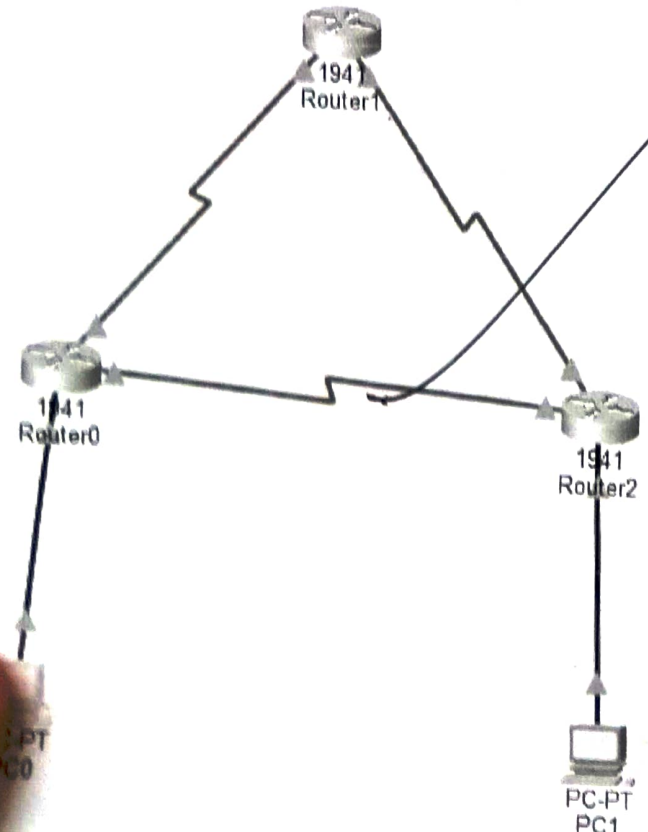
Aim:

To design and configure an internetwork using a switch, router and ethernet cables and verify connectivity between computers.

Procedure:

* Open cisco packet tracer and place computers.
* switch and router in the workspace
* Connect the computer to the switch using ethernet cables.
* connect the switch to the router using an Ethernet cable.
* assign static IP address, subnet mask and default gateway address

ing command to
between computers.
successful transmission



12/10/20

network was successfully
igured using a switch,
net cables in cisco

EX.No: Interneworking with routers in Cisco Packet Tracer simulation

a) Design and configure a simple internetwork using router

Aim:-

To design and configure a simple internetwork using a router in cisco packet tracer and fast connectivity between two PCs.

Procedure:-

Step 1: configure router 1

* select the router and open CLI
* Press enter to start configuration

Router 1.

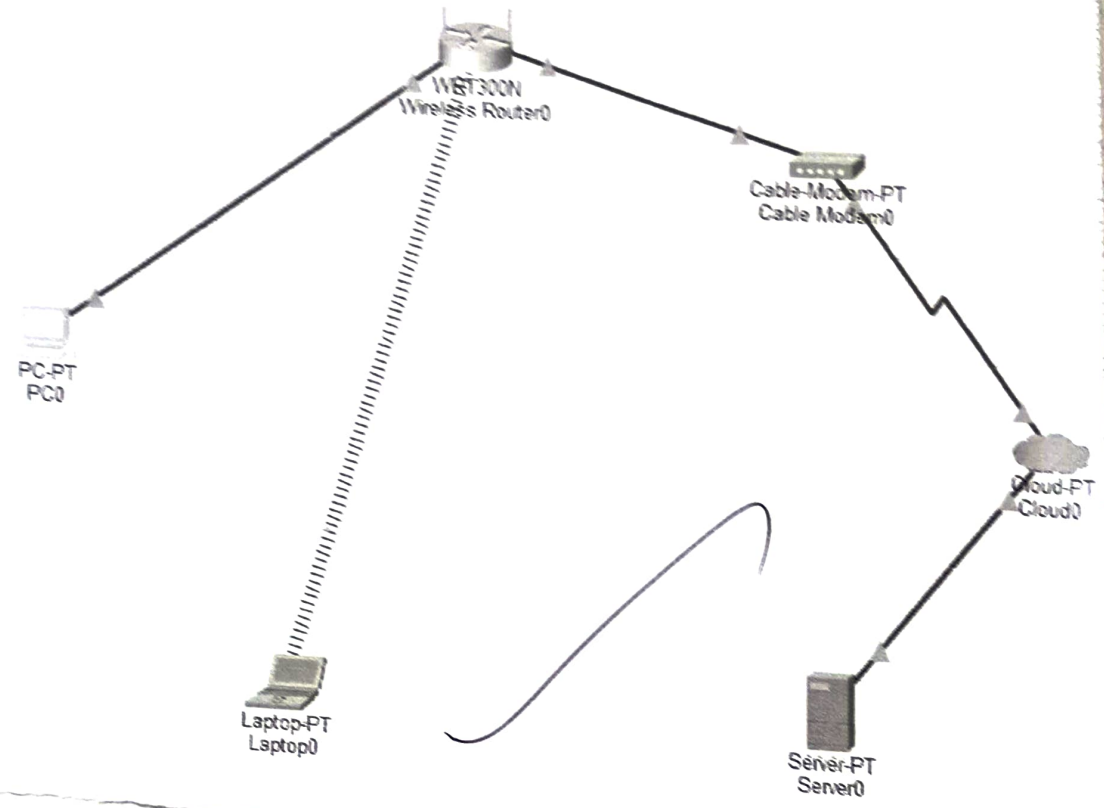
* Type enable to activate the Privileged mode.

Step 2 -

* Assign IP address to every PC in the network.
* select the PC, go to the desktop and select IP configuration

Step 3:

* connect fast ethernet 0 port of PC 0. with fast ethernet 0/0 port of router 1 using a copper straight-through cable.
* connect fast Ethernet 0 port of PC 1 with fast ethernet 0/1 port of Router 1 using a copper straight-through cable.



12/6/24

Result :-

The simple internetwork was successfully designed and configured using a router. connectivity between PC0 and PC1 was successfully executed.

b) Design and configure an interwork using wireless routers DHCP server and internet cloud

i

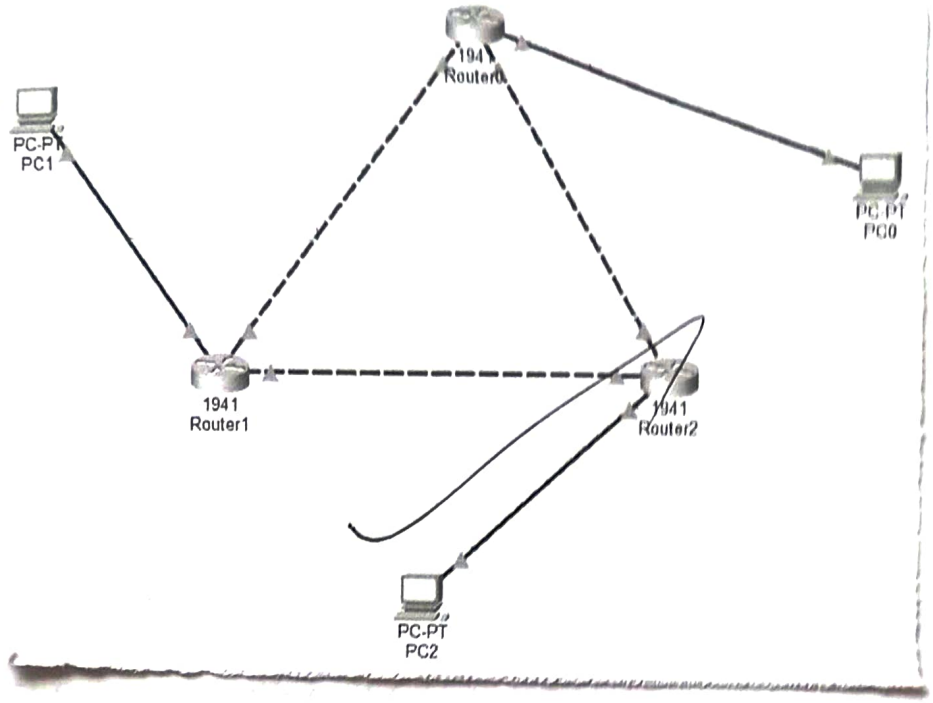
Aim:

To design and configure an interwork using a wireless router, DHCP server and internet cloud in cisco packet tracer.

Procedure:

* Open cisco packet tracer and create the required network topology.
* Connect PC, laptop, wireless router, cable modem, internet cloud server.
* Configure the wireless network set the PC to DHCP.
* Configure the server with IP 208.67.220.220 and enable DHCP and DNS.
* Verify connectivity using ipconfig and ping cisco.com.

Result



Result :-

12/12/22 the Internetwork was successfully designed and configured :